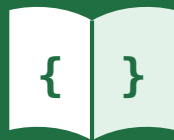

Module 17 — GraphQL and WebSocket

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 GraphQL in One Page	2
2.2 Schema-First	3
2.3 The N+1 Problem	3
2.4 WebSocket and STOMP	3
3. Step-by-Step Examples	3
3.1 The Schema	4
3.2 Queries: @QueryMapping	5
3.3 Nested Fields: @SchemaMapping and @BatchMapping	6
3.4 Mutations and Errors	7
3.5 Subscriptions	8
3.6 STOMP: Live Price Notifications	8
3.7 Testing	9
4. Common Mistakes and Best Practices	10
5. Summary	10
6. Further Reading	10

1. Learning Goals

By the end of this module you will be able to:

- Explain when GraphQL fits better than REST, and when it does not
- Write a GraphQL schema first and implement it with `@QueryMapping`, `@MutationMapping` and `@SchemaMapping`
- Recognise the N+1 problem and solve it with `@BatchMapping` (a `DataLoader`)
- Turn exceptions into useful GraphQL errors with `@GraphQLExceptionHandler`
- Push data to clients: GraphQL subscriptions and STOMP over WebSocket
- Test all of it with `GraphQLTester` and a STOMP client

Prerequisites: Module 06 (PostgreSQL), Module 13 (Reactor Flux basics) · **Estimated time:** 5 hours · **Docker required**

2. Concepts

2.1 GraphQL in One Page

With REST, the **server** decides the shape of each response: `GET /api/books/42` returns what the endpoint returns. With GraphQL there is **one endpoint** (`POST /graphql`), and the **client** writes a query that names exactly the fields it needs:

```
{ book(isbn: "9780134685991") { title author { name } } }
```

```
{ "data": { "book": { "title": "Effective Java", "author": { "name": "Joshua Bloch" } } } }
```

Operation	Purpose	Transport in this module
query	read	HTTP POST /graphql
mutation	change	HTTP POST /graphql
subscription	a stream of results	WebSocket /graphql

GraphQL is a good fit when many different clients (web, mobile, partners) need different views of the same connected data. For simple CRUD, file downloads or HTTP caching, REST remains simpler.

2.2 Schema-First

The schema (`src/main/resources/graphql/*.graphqls`) is the contract. Spring for GraphQL reads it at startup and connects every field to a **data fetcher**. Annotated controller methods become data fetchers, and the method name is the field name. A field without a method is read from the property with the same name of the parent object (`Book.title() → title`).

2.3 The N+1 Problem

A list of N books with the author of each: 1 query for the books, then N queries for the authors. A **DataLoader** collects all keys requested in one round of execution and loads them together. With `@BatchMapping`, Spring for GraphQL registers the DataLoader for you.

2.4 WebSocket and STOMP

WebSocket is a full-duplex connection that stays open, so the server can send at any time. WebSocket itself only moves frames. **STOMP** adds a small messaging protocol on top: `SUBSCRIBE /topic/prices`, `SEND /app/..., MESSAGE`. Spring's **simple broker** keeps the subscriptions in memory and delivers every message to all subscribers of a destination.

	GraphQL subscription	STOMP
Client asks for	a GraphQL query with selected fields	a destination (<code>/topic/prices</code>)
Protocol	<code>graphql-transport-ws</code>	STOMP frames
Good for	clients that already speak GraphQL	notifications, chat, dashboards

3. Step-by-Step Examples

With Docker running, start the application. PostgreSQL starts from the root `compose.yaml`, and Flyway creates the schema `graphql` with four books, three authors and three reviews:

```
./mvnw -pl modules/17-graphql-websocket/lesson spring-boot:run
```

The tour output (shortened):

```
=== 3.2 A query: only the fields we ask for ===
  titles: [Designing Data-Intensive Applications, Effective Java, Java Puzzlers, Spring
↪   in Action]
=== 3.3 N+1: author via @BatchMapping, reviews via @SchemaMapping ===
  4 books → author queries: 1, review queries: 4
=== 3.4 A mutation, then one with a wrong number of stars ===
  pushed to the subscriber: Review[id=4, isbn=9780321336781, stars=4, text=Tricky!]
  added: {id=4, stars=4, text=Tricky!}
  error: [{message=Stars must be between 1 and 5, not 9, ...
↪   extensions={classification=BAD_REQUEST}}]
```

Then open <http://localhost:8080/graphql> for an in-browser IDE, and run the requests in [requests.http](#).

3.1 The Schema

```
type Query {
  books: [Book!]!
  book(isbn: ID!): Book
}

type Mutation {
  addReview(input: ReviewInput!): Review!
}

type Subscription {
  reviewAdded(isbn: ID!): Review!
}

type Book {
  isbn: ID!
  title: String!
  price: Float!
  author: Author!
  reviews: [Review!]!
}

type Author {
  id: ID!
  name: String!
}

type Review {
  id: ID!
  isbn: ID!
```

```

    stars: Int!
    text: String!
}

input ReviewInput {
    isbn: ID!
    stars: Int!
    text: String!
}

```

- ! means non-null. [Review!]! is a list that is never null and contains no nulls.
- book(isbn: ID!): Book has no !: an unknown ISBN returns null, not an error.
- input ReviewInput is a type for arguments. Output types (Review) and input types are separate in GraphQL.

The configuration (under spring: in application.yaml) enables GraphiQL and the WebSocket endpoint for subscriptions:

```

graphql:
  graphiql:
    enabled: true                # a browser IDE at /graphiql (development only)
  websocket:
    path: /graphql              # subscriptions over WebSocket, same path as HTTP

```

! Warning

Enable GraphiQL only in development. In production, also think about limiting query depth and complexity: a client can write a very expensive query.

3.2 Queries: @QueryMapping

```

@QueryMapping                                // Query.books
List<Book> books() {
    return catalog.findAllBooks();
}

@QueryMapping                                // Query.book(isbn: ID!) – null when
↪ unknown
@Nullable Book book(@Argument String isbn) {
    return catalog.findBook(isbn).orElse(null);
}

```

@QueryMapping is a shortcut for @SchemaMapping(typeName = "Query"). @Argument binds the argument with the same name. The Book record has isbn, title and price, so these fields need no method.

Over HTTP, every request is a POST /graphql with a JSON body. When a field fails (for example a validation error in a resolver), the response is still 200 OK, and the errors are in the errors array. GraphQLoverHttpTest sends a real HTTP request with HttpGraphQLTester.

3.3 Nested Fields: @SchemaMapping and @BatchMapping

Book has no Author object, only authorId. The field Book.author needs a data fetcher, and the lesson solves it in the efficient way:

```
/** Book.author for ALL books of the result in one call: no N+1. */
@BatchMapping
Map<Book, Author> author(List<Book> books) {
    Set<Long> ids = books.stream().map(Book::authorId).collect(Collectors.toSet());
    Map<Long, Author> authors = catalog.findAuthors(ids).stream()
        .collect(Collectors.toMap(Author::id, Function.identity()));
    return books.stream().collect(Collectors.toMap(Function.identity(), book ->
        ↪ authors.get(book.authorId())));
}
```

The method receives **all** books of the result at once and returns one author per book. The repository needs one IN query:

```
/** One query for many authors: this is what @BatchMapping calls once per request. */
public List<Author> findAuthors(Collection<Long> ids) {
    authorQueries.incrementAndGet();
    return jdbc.sql("SELECT id, name FROM author WHERE id IN (:ids)")
        .param("ids", ids)
        .query(Author.class)
        .list();
}
```

Book.reviews uses the simple way on purpose, so that the difference is visible:

```
/** Book.reviews, called once PER book: simple, but N+1 queries for a list of N books. */
@SchemaMapping
List<Review> reviews(Book book) {
    return catalog.findReviews(book.isbn());
}
```

BookQueryTest.batchMappingLoadsAllAuthorsInOneCallButReviewsNeedOneCallPerBook counts the calls: one author query, but at least four review queries for four books.

★ Tip

@SchemaMapping is fine for a field of a single object. Use @BatchMapping as soon as the field appears in lists.

3.4 Mutations and Errors

```
@MutationMapping
Review addReview(@Argument ReviewInput input) {
    return reviews.add(input);
}

@GraphQLExceptionHandler // like @ExceptionHandler, but produces
    ↪ a GraphQL error
GraphQLError invalidReview(InvalidReviewException e, DataFetchingEnvironment env) {
    return GraphQLError-
        ↪ Builder.newError(env).errorType(ErrorType.BAD_REQUEST).message(e.getMessage()).build();
}

@GraphQLExceptionHandler
GraphQLError bookNotFound(BookNotFoundException e, DataFetchingEnvironment env) {
    return GraphQLError-
        ↪ Builder.newError(env).errorType(ErrorType.NOT_FOUND).message(e.getMessage()).build();
}
```

- @Argument ReviewInput input binds the input object to the record.
- The service throws normal Java exceptions:

```
public Review add(ReviewInput input) {
    if (input.stars() < 1 || input.stars() > 5) {
        throw new InvalidReviewException("Stars must be between 1 and 5, not " +
            ↪ input.stars());
    }
    if (catalog.findBook(input.isbn()).isEmpty()) {
        throw new BookNotFoundException(input.isbn());
    }
    Review review = catalog.insertReview(input);
    // two requests may add reviews at the same time: retry briefly instead of failing on
    ↪ a concurrent emit
    added.emitNext(review, Sinks.EmitFailureHandler.busyLooping(Duration.ofMillis(100)));
    return review;
}
```

- @GraphQLExceptionHandler is the GraphQL counterpart of @ExceptionHandler. It turns the exception into an entry in the errors array of the response, with a classification such as BAD_REQUEST or NOT_FOUND. An exception without a handler becomes

INTERNAL_ERROR with a generic message: details of unexpected errors are not given to the client.

The same handlers can be put into a `@ControllerAdvice` class to apply to all controllers.

3.5 Subscriptions

A subscription method returns a Flux. Every element becomes one result for the client:

```
@SubscriptionMapping
Flux<Review> reviewAdded(@Argument String isbn) {
    return reviews.reviewsAdded().filter(review -> review.isbn().equals(isbn));
}
```

The reviews come from a Reactor **sink**, a Flux that code can push into:

```
// A hot stream: every subscriber gets the reviews added after it subscribed (no history,
↪ no buffer).
private final Sinks.Many<Review> added = Sinks.many().multicast().directBestEffort();
```

A subscriber receives only the reviews added after it subscribed. `ReviewSubscriptionTest` subscribes, adds reviews for two books and receives only the ones for its book.

i Note

The sink lives in the memory of one application instance. With several instances, a review added on instance A does not reach the subscribers of instance B. Then the events have to go through a broker (for example Kafka, module 11, or Redis pub/sub).

3.6 STOMP: Live Price Notifications

The configuration registers the WebSocket endpoint and the broker:

```
@Configuration
@EnableWebSocketMessageBroker
class WebSocketConfiguration implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws"); // clients connect to
        ↪ ws://host:8080/ws
    }

    @Override
```

```

    public void configureMessageBroker(MessageBrokerRegistry broker) {
        broker.enableSimpleBroker("/topic");           // server → clients:
        ↪ /topic/...
        broker.setApplicationDestinationPrefixes("/app"); // clients →
        ↪ @MessageMapping methods: /app/...
    }
}

```

Any bean can send to a destination with `SimpMessagingTemplate`. Here a normal REST endpoint changes a price and notifies all subscribers:

```

@PutMapping("/api/books/{isbn}/price")
@ResponseStatus(HttpStatus.NO_CONTENT)
void changePrice(@PathVariable String isbn, @Valid @RequestBody PriceChangeRequest
    ↪ request) {
    BigDecimal price = request.price();
    int updated = jdbc.sql("UPDATE book SET price = ? WHERE isbn = ?").params(price,
        ↪ isbn).update();
    if (updated == 0) {
        throw new ResponseStatusException(HttpStatus.NOT_FOUND, "No book with ISBN " +
            ↪ isbn); // → ProblemDetail
    }
    messaging.convertAndSend("/topic/prices", new PriceChanged(isbn, price)); // JSON
    ↪ to all subscribers
}

```

Try it in the browser: open `http://localhost:8080/prices.html` (a small page with the `@stomp/stompjs` client), then run the price change in `requests.http`. The new price appears without reloading the page.

The prefix `/app` is for the other direction: a client sends a message to `/app/...`, and a `@MessageMapping` method in a `@Controller` handles it. The exercises only need the server → client direction.

3.7 Testing

- `@AutoConfigureGraphQLTester` gives a `GraphQLTester` that runs documents directly against the GraphQL engine, without HTTP. `path("books[*].title")` selects values with `JsonPath`, and `errors().expect(...)` checks errors.
- `@AutoConfigureHttpGraphQLTester` with `RANDOM_PORT` sends real HTTP requests.
- `executeSubscription().toFlux(...)` returns the subscription as a `Flux`. Check it with `StepVerifier`.
- `STOMP: WebSocketStompClient` connects to the running server. The `SUBSCRIBE` frame is asynchronous, so the test repeats the action with `Awaitility` until the first message arrives (`PriceNotificationTest`).

4. Common Mistakes and Best Practices

!! Caution

A field that is resolved per parent object runs once for every row of a list. Watch the query count (as the tests here do) before a list of 100 books becomes 101 queries.

- **Do:** design the schema for the clients, not as a copy of the tables. `author: Author!` instead of `authorId: ID!`.
- **Don't:** return internal exception messages to clients. Map known exceptions with `@GraphQLExceptionHandler`, and let the rest become `INTERNAL_ERROR`.
- **Do:** make a field nullable when “not found” is a normal answer (`book(isbn): Book`).
- **Don't:** rely on HTTP status codes for errors. Errors in fields come back with 200 and an errors array, so clients must always check errors.
- **Do:** secure the WebSocket endpoint like any other endpoint (module 12), and limit who may subscribe to which destination.
- **Don't:** forget that the simple broker and the sink are in-memory. For several instances, use a broker relay (RabbitMQ, ActiveMQ) or a message system.

5. Summary

- A GraphQL schema is the contract. The client selects fields, and there is one endpoint, `/graphql`.
- `@QueryMapping`, `@MutationMapping` and `@SubscriptionMapping` implement the root fields. `@SchemaMapping` implements a field of a type.
- `@BatchMapping` loads a field for all parent objects in one call and solves N+1.
- `@GraphQLExceptionHandler` turns exceptions into classified GraphQL errors.
- Subscriptions return a Flux and run over WebSocket. STOMP adds destinations and a broker for notifications.

Next step: [Exercises](#)

6. Further Reading

- [Spring for GraphQL Reference](#) · [Annotated Controllers](#) · [Testing](#)
- [Spring Boot — Spring for GraphQL](#)
- [Spring Framework — WebSockets and STOMP](#)
- [GraphQL — Learn](#)
- [STOMP 1.2 Specification](#)